

Функциональное программирование на Haskell



<https://forms.gle/Sxt7Jw8frWWoDBq47>



<https://t.me/+0t6oV2i178ZlMjdi>



Структура курса

Лекции

- 1 - вы сейчас тут
- 2-4 - основы языка
- 5-7 - монады
- 8 - ?

Практики

- Только лабораторные
- 1-4 - без защиты
- 5,6 - с защитой
- Неделя дедлайна = -20%

Если ничего не понятно по системе оценивания:
<https://github.com/is-fp-y27/.github/blob/main/profile/rules/points.md>

```
1  class Validator {
2      static void validate(Map<String, User> users) {
3          if (users.containsKey("admin") && !users.get("admin").isActive()) {
4              users.remove("admin"); // валидация мутирует входные данные
5          }
6      }
7  }
8
9  Map<String, User> users = new HashMap<>();
10 users.put("admin", new User("Alice", false));
11 Validator.validate(users);
12 User admin = users.get("admin"); // admin == null
```

Иммутабельность by default



```
1  users    = [("admin", User ("Alice", False))]  
2  users'   = validate users  
3  result1  = lookup "admin" users'           -- Nothing  
4  result2  = lookup "admin" users            -- "Alice"  
_
```

```
1  double withdraw(BankAccount account, double amount) {
2      if (amount > account.getBalance()) {
3          logger.error("Not enough money for account {}", account.getId());
4          throw new IllegalStateException("Insufficient funds");
5      }
6      account.setBalance(account.getBalance() - amount);
7      logger.info("Withdraw {} from {}", amount, account.getId());
8      auditService.save(account); // ещё и БД
9      return account.getBalance();
10 }
```

Чистые функции. Отсутствие Side Effects



```
1  newBalance = calcWithdraw balance amount
2  -- логи отдельно
3  -- работа с бд отдельно
```

Свойства и преимущества чистых функций



1. Прозрачность по ссылке (referential transparency)

- Любое выражение можно заменить его результатом, не меняя поведение программы
- `mult(a, b) = a * b`

Свойства и преимущества чистых функций



1. Прозрачность по ссылке (referential transparency)

- Любое выражение можно заменить его результатом, не меняя поведение программы
- `mult(a, b) = a * b`

2. Подстановка тела функции вместо вызова (inlining)

- Уменьшение накладных расходов на вызов
- Применение других оптимизаций

Свойства и преимущества чистых функций



1. Прозрачность по ссылке (referential transparency)

- Любое выражение можно заменить его результатом, не меняя поведение программы
- `mult(a, b) = a * b`

2. Подстановка тела функции вместо вызова (inlining)

- Уменьшение накладных расходов на вызов
- Применение других оптимизаций

3. Переупорядочивание вычислений

- Отложить вычисление
- Удалить повторы

Свойства и преимущества чистых функций



1. Прозрачность по ссылке (referential transparency)

- Любое выражение можно заменить его результатом, не меняя поведение программы
- `mult(a, b) = a * b`

2. Подстановка тела функции вместо вызова (inlining)

- Уменьшение накладных расходов на вызов
- Применение других оптимизаций

3. Переупорядочивание вычислений

- Отложить вычисление
- Удалить повторы

4. Мемоизация

Ленивые вычисления



```
1  public int firstArgument(int a, int b) {  
2      return a;  
3  }  
4  
5  firstArgument(1, 1/0); // ArithmeticException
```

Ленивые вычисления



```
1  public int firstArgument(int a, int b) {  
2      return a;  
3  }  
4  
5  firstArgument(1, 1/0); // ArithmeticException
```

```
1  firstArgument a b = a  
2  result = firstArgument (1) (1/0) -- ok
```

1. Иммутабельность
2. Чистые функции
3. Ленивые вычисления (не во всех языках)
4. Функции высших порядков
5. Алгебраические типы данных (не во всех языках)

State of Haskell

[Вакансии](#)[Резюме](#)[Компании](#)[Сохранить поиск](#)[Найти](#)[+ Регион](#)[В названии вакансии](#)[Фильтры](#)

Найдена 1 вакансия «haskell разработчик»

По соответствию ▾ За всё время ▾ 50 вакансий ▾

[Вакансии на карте](#)

По вашему запросу ещё будут появляться новые вакансии. Присылать вам?

haskell разработчик • В названии вакансии

[В мессенджер](#)[На почту](#)

Сейчас смотрят 8 человек



Middle backend-разработчик Haskell

180 000 – 220 000 ₽ за месяц, на руки

Опыт 1-3 года

Можно удалённо

Выплаты: Два раза в месяц

getads

★ 4.3 • 4 отзыва

Москва • Курская и еще 3 • • •

Разрабатывать и развивать backend-сервисы платформы на **Haskell**. Моделировать данные и вести миграции (Postgres), работать со справочными и иерархическими данными.

Любовь к функциональным языкам и понимание их основ. Обязательный опыт с **Haskell** (pet-проекты и open source тоже считаются).

[Откликнуться](#)[Связаться](#)

да / сильная поддержка
 нет
 частично / иначе / ?

Язык	Чистый	Ленивые вычисления	Типизация	Абстрактные типы данных	Иммутабельность	Классы типов
Common Lisp	нет	через thunks	динамическая	да	нет	?
Clojure	нет	да	динамическая	да	да	нет
F#	нет	да	статическая	да	да	нет
Haskell	да	по умолчанию	статическая	да	да	да
Scala	нет	да	статическая	да	да	да
JavaScript	нет	расширением	динамическая	расширением	частично	?
Miranda	да	по умолчанию	статическая	да	да	нет
Erlang	нет	нет	динамическая	да	да	?
Python	нет	генераторы	динамическая	да	частично	?
Kotlin	нет	Sequence	статическая	да	да	нет
Swift	нет	нет	статическая	да	да	нет
Rust	нет	итераторы	статическая	да	да	через traits
D	опционально	опционально	статическая	?	да	нет

Где использовать?

Лучшие применения

- Компиляторы, языки
- Конкурентность
- Парсинг /
форматирование
- CLI
- Backend

Плохие применения

- Data Science, ML
- GUI
- GameDev

GHC и GHCi

- **GHC** (Glasgow Haskell Compiler) – компилятор Haskell
- **GHCi** – интерактивный интерпретатор (REPL)
- Основные команды GHCi:
 - `ghci` – вход в REPL
 - `:quit` – выход
 - `:load File.hs` / `:l File.hs` – загрузить модуль
 - `:reload` / `:r` – перезагрузить

Настоятельно рекомендую установку через GHCup <https://www.haskell.org/ghcup/>

cabal init

```

└─ cabal
  └─ app
      Main.hs
      cabal-proj.cabal
      CHANGELOG.md
      LICENSE
```

VS

stack new stack-proj

```

└─ stack
  └─ stack-proj
      .stack-work
      app
          Main.hs
      src
          Lib.hs
      test
          Spec.hs
      .gitignore
      CHANGELOG.md
      LICENSE
      package.yaml
      README.md
      Setup.hs
      stack.yaml
      stack.yaml.lock
      stack-proj.cabal
```

cabal

stack

система сборки и пакетов

роль

обёртка над cabal

.cabal или *cabal.project*

конфигурация

stack.yaml, *package.yaml*

cabal repl

REPL

stack ghci

cabal build

build

stack build

cabal run

run

stack run

не был, но стал

канон?

был, но перестал



ВСЁ!